



Labforweb

nerd academy

Python

Origini di Python



Python è stato creato da **Guido van Rossum** alla fine degli anni '80

Il primo rilascio ufficiale è avvenuto nel **1991** (versione 0.9.0)

Ispirato a linguaggi come ABC e il più noto C, con focus sulla leggibilità e semplicità

Il nome "Python" deriva dal gruppo comico britannico Monty Python

Evoluzione di Python



Alcune date utili

1994: Rilascio di Python 1.0 con moduli, eccezioni e funzioni di base

2000: Python 2.0 introduce la gestione automatica della memoria

2008: Python 3.0 porta miglioramenti alla sintassi e Unicode nativo

Oggi: Python è uno dei linguaggi più usati in AI, Data Science, Web e Automazione

Cos'è Python?



- » **Python** è un linguaggio di programmazione ad alto livello
- » È interpretato, dinamico e orientato agli oggetti
- » Progettato per essere semplice da leggere e scrivere
- » Ampiamente utilizzato in sviluppo web, data science, AI, automazione e altro

Vantaggi di Python



- » Sintassi chiara e leggibile
- » Grande community e supporto
- » Ampia gamma di librerie e framework
- » Multipiattaforma e facile da integrare con altri linguaggi
- » Utilizzato da aziende leader come Google, Facebook, Netflix, ecc.

Perché usare Python?



- » Perfetto per principianti e professionisti
- » Ideale per prototipazione rapida e sviluppo veloce
- » Supportato da un ecosistema ricco di strumenti
- » Applicabile in molte aree: Web Development, AI, Machine Learning, Automazione, ecc.

Installazione di Python



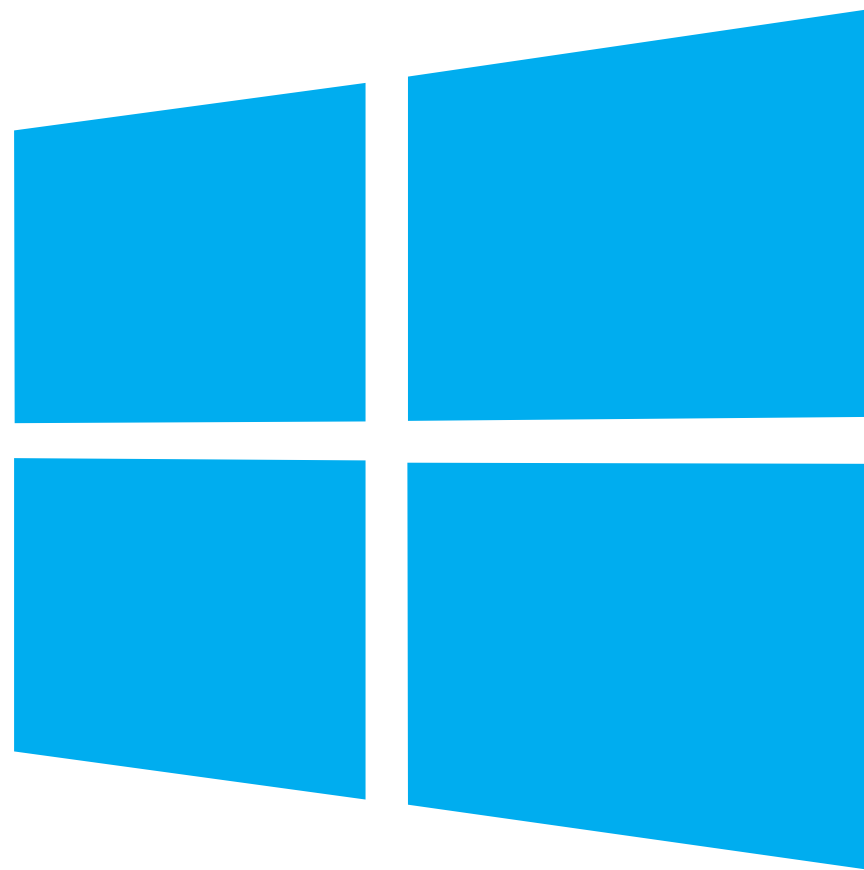
- » Python è disponibile per Windows, macOS e Linux
- » Scaricabile dal sito ufficiale: <https://www.python.org/downloads/>

N.B.

- » Assicurarsi di scaricare la versione più recente
- » Seguire le istruzioni specifiche per ogni sistema operativo



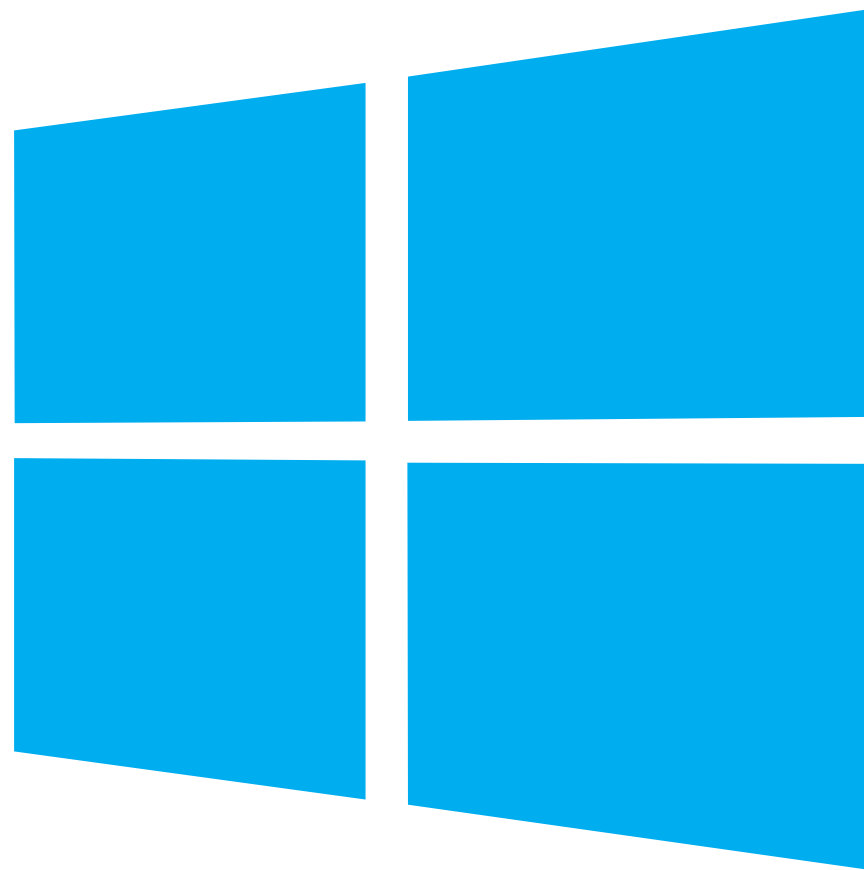
Installazione di Python su Windows – Download



1. Vai su <https://www.python.org/downloads/>
2. Clicca su **Download Python [versione]**
3. Apri il file **.exe** scaricato
4. Seleziona l'opzione **Add Python to PATH**
5. Clicca su **Install Now** e attendi il completamento



Verifica dell'installazione su Windows

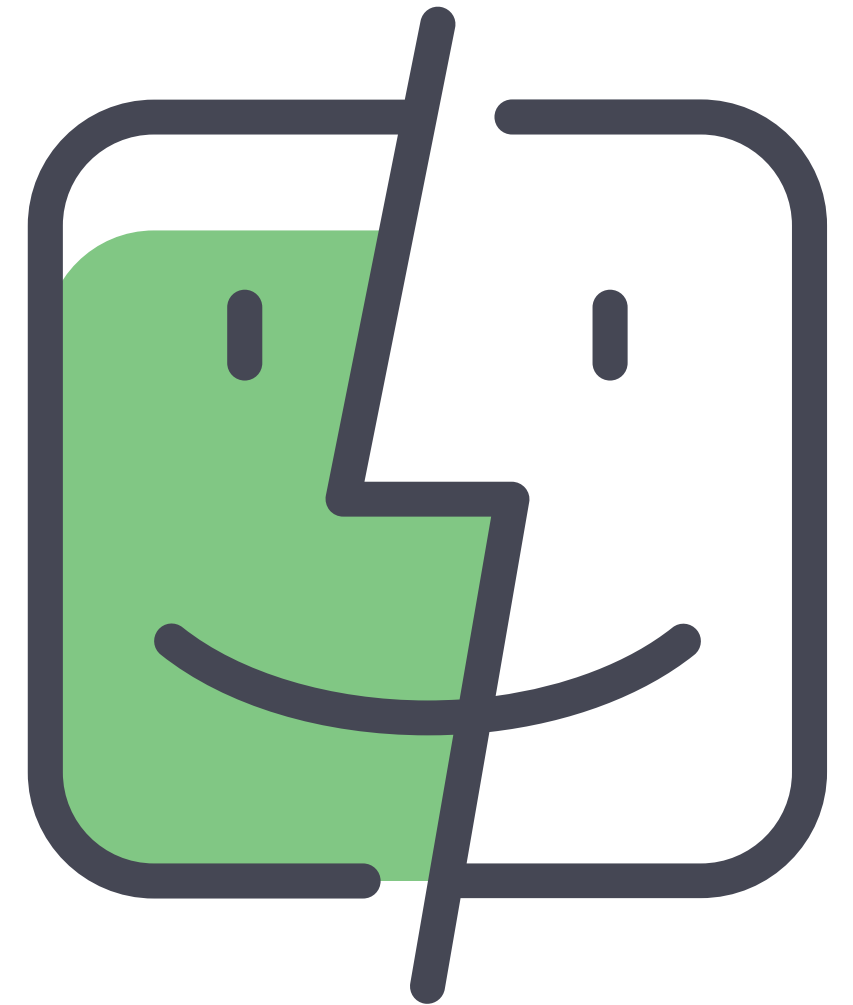


1. Apri il **Prompt dei comandi** (cmd)
2. Digita `python --version` e premi Invio
3. Se Python è installato correttamente, vedrai la versione installata
4. Per accedere alla shell interattiva, digita `python` e premi Invio

Installazione di Python su macOS



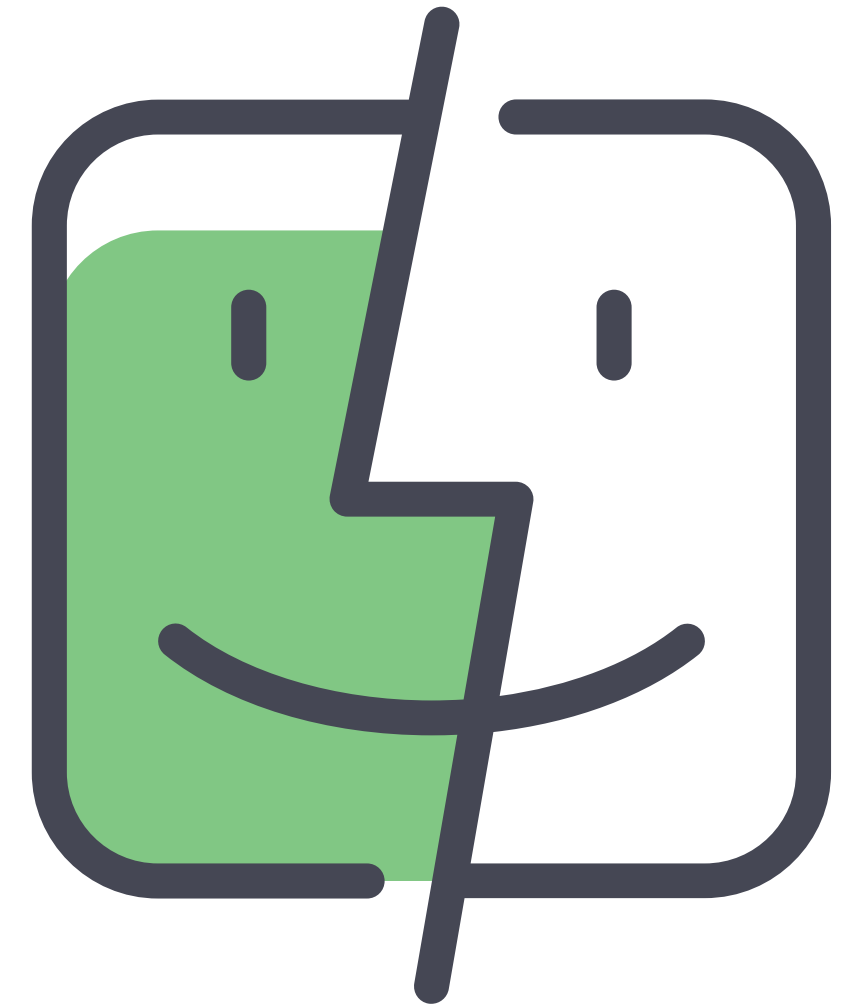
1. Vai su <https://www.python.org/downloads/>
2. Clicca su **Download Python [versione]**
3. Apri il file **.pkg** scaricato e segui le istruzioni
4. Completa l'installazione accettando le impostazioni predefinite



Verifica dell'installazione su macOS



1. Apri il **Terminale**
2. Digita `python3 --version` e premi Invio
3. Se Python è installato correttamente, vedrai la versione installata
4. Per accedere alla shell interattiva, digita `python3` e premi Invio





Installazione alternativa con Homebrew (macOS)

1. Apri il **Terminale**

2. Installa **Homebrew** (se non già installato):

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

3. Installa Python con il comando: `brew install python`

4. Verifica l'installazione con `python3 --version`

Le Variabili in Python



Una variabile è un contenitore per un valore

In Python, non è necessario dichiarare il tipo di una variabile

L'assegnazione avviene con =

Esempio:



```
nome = 'Alice'  
età = 25  
altezza = 1.75
```


Tipi di Dati in Python



- » **Numerici:** int, float, complex
- » **Testo:** str
- » **Booleani:** bool (True o False)
- » **Liste:** list (collezione ordinata e modificabile)
- » **Tuple:** tuple (collezione ordinata e immutabile)
- » **Dizionari:** dict (collezione di coppie chiave-valore)
- » **Set:** set (collezione non ordinata di elementi unici)





Dichiarazione e Conversione di Tipi di Dati

Python consente la conversione tra tipi di dati

Esempio:



```
x = 5      # int  
y = float(x) # Convertito in float  
z = str(x)  # Convertito in stringa
```

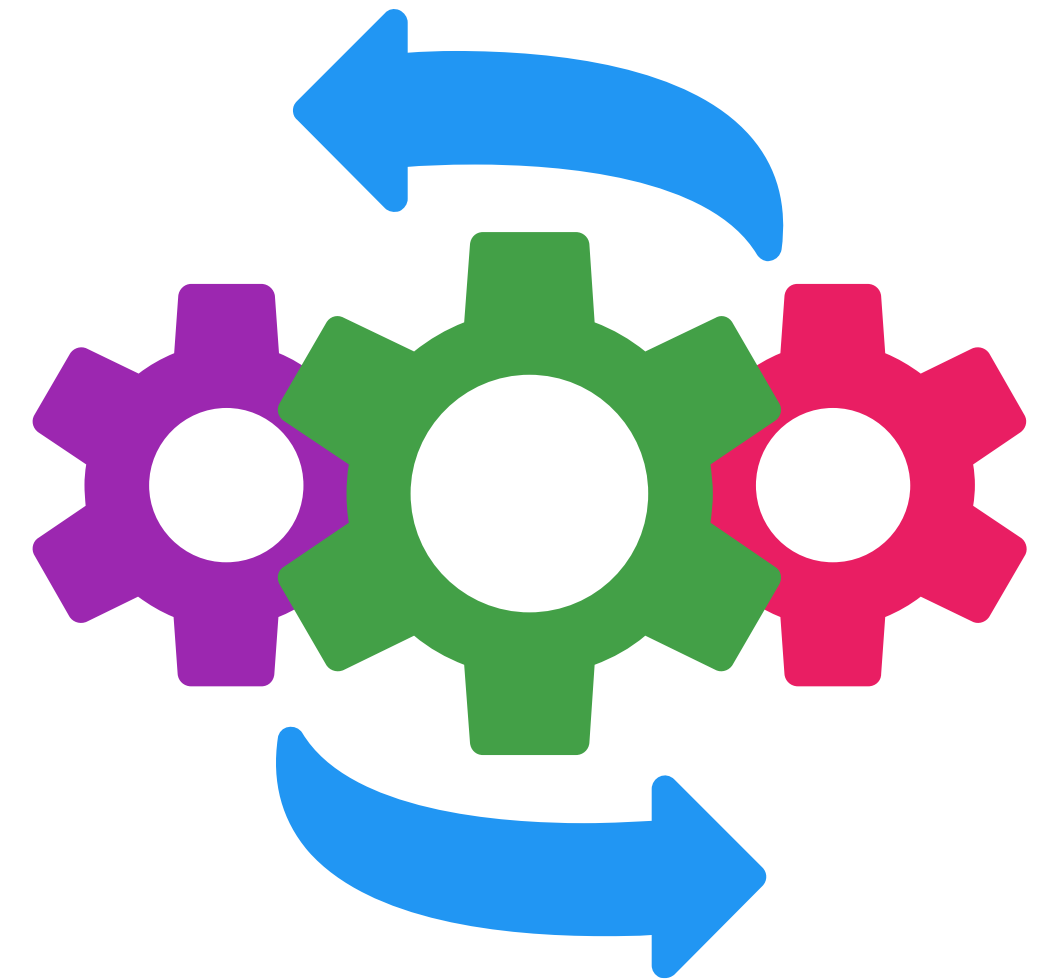
Verifica del tipo con **type()**:

```
print(type(x))
```

Operatori in Python



- » **Aritmetici:** +, -, *, /, //, %, **
- » **Assegnazione:** =, +=, -=, *=, /=, //=
- » **Confronto:** ==, !=, >, <, >=, <=
- » **Logici:** and, or, not
- » **Appartenenza:** in, not in
- » **Identità:** is, is not



Esempi di Operatori Aritmetici



Esempi di operazioni aritmetiche in Python:

- `a = 10`
- `b = 3`
- `somma = a + b` `# 13`
- `differenza = a - b` `# 7`
- `prodotto = a * b` `# 30`
- `divisione = a / b` `# 3.33`
- `modulo = a % b` `# 1` (resto della divisione)
- `potenza = a ** b` `# 1000`



Esempi di Operatori Logici e di Confronto

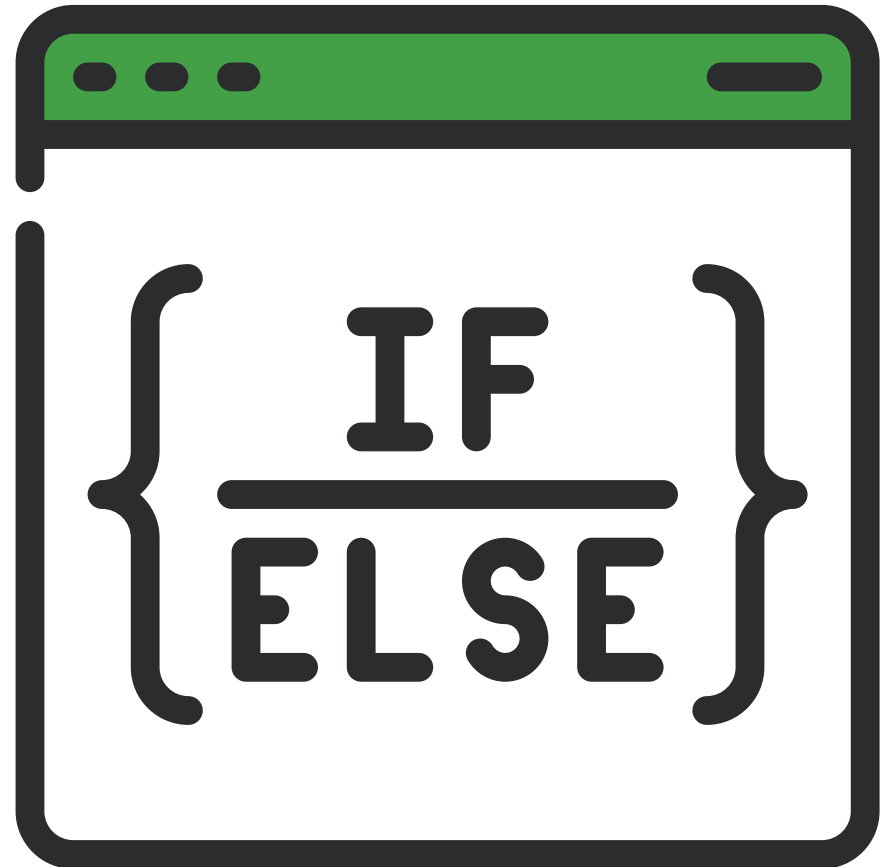
“

- `x = 5`
- `y = 10`
- `print(x > y) # False`
- `print(x == y) # False`
- `print(x != y) # True`

- `a = True`
- `b = False`
- `print(a and b) # False`
- `print(a or b) # True`
- `print(not a) # False`

”

Strutture di Controllo in Python: if, else, elif



In Python, le strutture di controllo **if**, **else** ed **elif** consentono di eseguire blocchi di codice diversi a seconda delle condizioni specificate.

Questi costrutti sono fondamentali per la logica di un programma.



Il costrutto if

L'istruzione **if** consente di eseguire un blocco di codice solo se una condizione è vera

Sintassi:

if condizione:

codice da eseguire se la condizione è vera

Esempio:



```
x = 10
```

```
if x > 5:
```

```
    print('x è maggiore di 5')
```





Il costrutto if-else

Se la condizione dell'if non è vera, possiamo usare **else** per eseguire un codice alternativo.

Sintassi:

if condizione:

codice eseguito se la condizione è vera

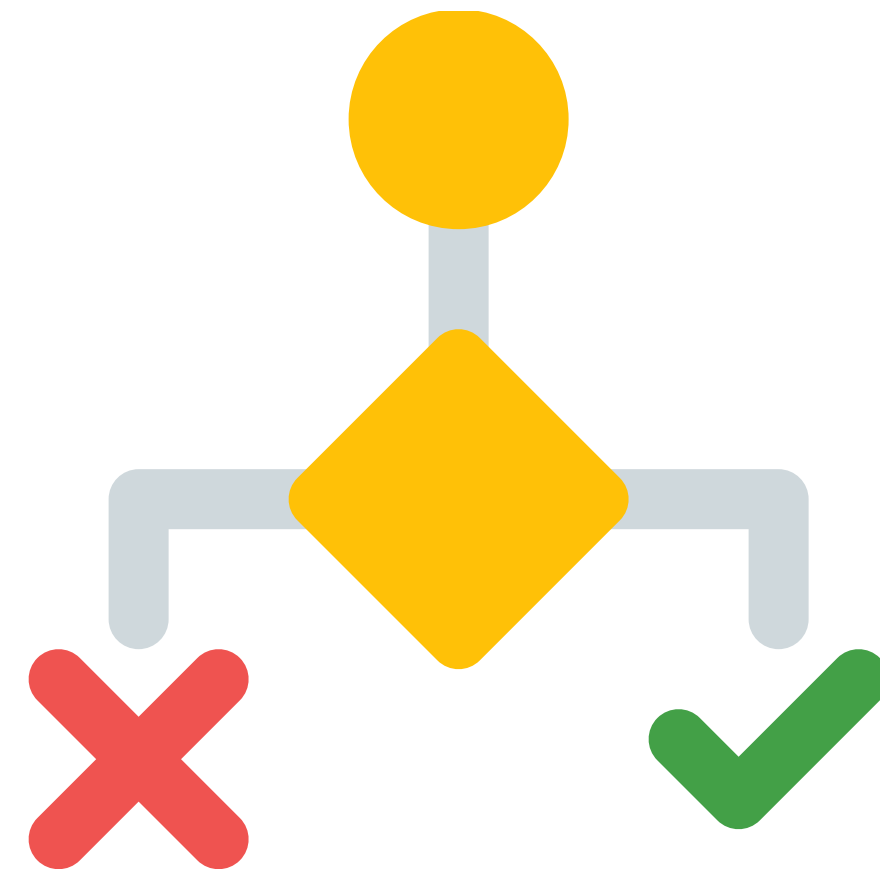
else:

codice eseguito se la condizione è falsa

Esempio:



```
x = 3
if x > 5:
    print('x è maggiore di 5')
else:
    print('x non è maggiore di 5')
```



Il costrutto if-elif-else



Quando abbiamo più condizioni da controllare, possiamo usare `elif`

Sintassi:

```
```python
if condizione1:
 # codice se condizione1 è vera
elif condizione2:
 # codice se condizione2 è vera
else:
 # codice se nessuna condizione è vera
```

**Esempio:**

```
```python
x = 10
if x < 0:
    print('Numero negativo')
elif x == 0:
    print('Numero uguale a zero')
else:
    print('Numero positivo')
```



If nidificati

È possibile **annidare** un'istruzione if all'interno di un'altra if.

Esempio:



```
x = 10
if x > 0:
    print('Numero positivo')
    if x > 5:
        print('Ed è maggiore di 5')
```

Operatori di confronto e logici



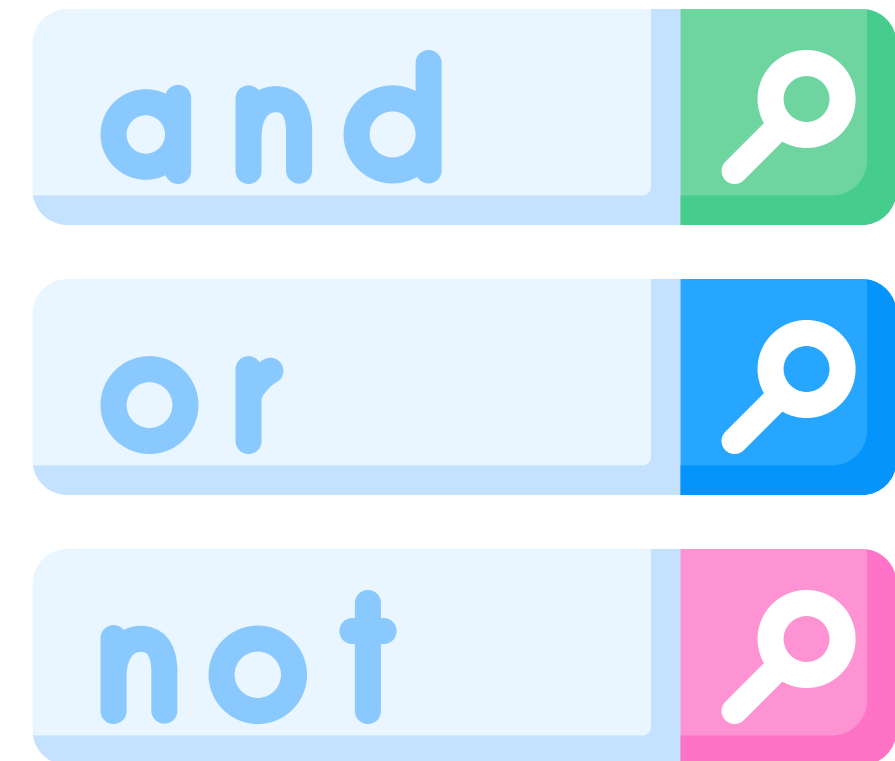
Operatori di confronto: ==, !=, >, <, >=, <=

Operatori logici: and, or, not

Esempio di utilizzo:



```
x = 8
if x > 5 and x < 10:
    print('x è compreso tra 5 e 10')
```





Le Tuple in Python

Le tuple sono un tipo di dato in Python simile alle liste, ma con una differenza fondamentale: sono immutabili, cioè il loro contenuto non può essere modificato dopo la creazione.

Le tuple sono utili per immagazzinare dati che non devono essere alterati durante l'esecuzione del programma.

```
t = ( 1 , 2 , 'python' , tuple() , {42, 'hi'} )
```

Diagram illustrating tuple indexing for the tuple `t`:

- `t[0]` points to `1`
- `t[1]` points to `2`
- `t[2]` points to `'python'`
- `t[3]` points to `tuple()`
- `t[4]` points to `{42, 'hi'}`

Una tupla può essere creata utilizzando le parentesi tonde `()`.

Esempi:

```
tupla_vuota = ()  
tupla_numeri = (1, 2, 3, 4, 5)  
tupla_mista = ('Python', 3.9, True)
```

Accesso agli Elementi



Gli elementi della tupla possono essere accessibili tramite gli indici

L'indicizzazione parte da 0.

Esempio:



```
tupla = ('Python', 'Java', 'C++')  
print(tupla[0]) # Output: Python  
print(tupla[-1]) # Output: C++
```

Operazioni di Base sulle Tuple



Concatenazione:

```
tupla1 = (1, 2, 3)
tupla2 = (4, 5, 6)
tupla3 = tupla1 + tupla2
print(tupla3) # Output: (1, 2, 3, 4, 5, 6)
```

Ripetizione:

```
tupla = ('A',) * 3
print(tupla) # Output: ('A', 'A', 'A')
```

Verifica presenza di un elemento:

```
tupla = (10, 20, 30)
print(20 in tupla) # Output: True
```

Immutabilità delle Tuple



Le tuple non possono essere modificate dopo la loro creazione

Esempio di errore quando si prova a modificare una tupla:



```
tupla = (1, 2, 3)
```

```
tupla[0] = 10 # Genererà un TypeError
```

Metodi Utili per le Tuple



count()

Conta il numero di volte che un elemento appare nella tupla

```
tupla = (1, 2, 2, 3, 2)
print(tupla.count(2)) # Output: 3
```

index()

Restituisce l'indice della prima occorrenza di un elemento

```
tupla = ('a', 'b', 'c')
print(tupla.index('b')) # Output: 1
```


Conversione tra Liste e Tuple



È possibile convertire una lista in una tupla e viceversa

Esempio:



```
lista = [1, 2, 3]
```

```
tupla = tuple(lista)
```

```
print(tupla) # Output: (1, 2, 3)
```

```
tupla = (4, 5, 6)
```

```
lista = list(tupla)
```

```
print(lista) # Output: [4, 5, 6]
```



Quando Usare le Tuple?

Se i dati devono rimanere immutabili, usare una tupla.

Quando si vuole migliorare le prestazioni rispetto alle liste.

Le tuple sono spesso utilizzate per restituire più valori da una funzione

Esempio di funzione che restituisce più valori:



```
def coordinate():  
    return (10, 20)  
  
x, y = coordinate()  
print(x, y) # Output: 10 20
```

I Dizionari in Python



Un dizionario si definisce utilizzando le parentesi graffe `{ }` e separando le chiavi dai valori con:

Esempio:



```
dizionario = {  
    'nome': 'Mario',  
    'età': 30,  
    'città': 'Roma'  
}  
print(dizionario) # Output: {'nome': 'Mario', 'età': 30, 'città': 'Roma'}
```


Accesso ai Valori di un Dizionario



Si può accedere ai valori di un dizionario usando le *chiavi*.

Esempio:



```
print(dizionario['nome']) # Output: Mario
```

Oppure con **get()**, che evita errori se la chiave non esiste:

```
print(dizionario.get('età')) # Output: 30
```

```
print(dizionario.get('indirizzo', 'Chiave non trovata')) # Output: Chiave non trovata
```

Aggiungere e Modificare Elementi



Per *aggiungere* un nuovo elemento, basta assegnare un valore a una nuova chiave:

```
dizionario['professione'] = 'Ingegnere'  
print(dizionario)
```

Per modificare un valore:

```
dizionario['età'] = 31  
print(dizionario)
```

Rimuovere Elementi



Si può rimuovere un elemento con **del** o con **pop()**:

```
del dizionario['città']
```

```
print(dizionario)
```

```
età = dizionario.pop('età') # Rimuove 'età' e restituisce il valore
```

```
print(età) # Output: 31
```

Metodi Principali dei Dizionari



keys()

Restituisce tutte le chiavi

```
print(dizionario.keys())
```

Output: dict_keys(['nome', 'professione'])

values()

Restituisce tutti i valori

```
print(dizionario.values())
```

Output: dict_values(['Mario', 'Ingegnere'])

items()

Restituisce coppie chiave-valore

```
print(dizionario.items())
```

*# Output: dict_items([('nome', 'Mario'),
('professione', 'Ingegnere')])*

Metodi Principali dei Dizionari



update()

Unisce due dizionari

```
dizionario.update({'nazionalità': 'Italiana'}) print(dizionario)
```

clear()

Svuota il dizionario

```
dizionario.clear() print(dizionario)
```

Output: {}



Iterare su un Dizionario



Si può iterare sulle chiavi, sui valori o sulle coppie chiave-valore.

» **Iterare sulle chiavi:**

```
for chiave in dizionario:  
    print(chiave)
```

» **Iterare sui valori:**

```
for valore in dizionario.values():  
    print(valore)
```

» **Iterare su chiavi e valori insieme:**

```
for chiave, valore in dizionario.items():  
    print(f'{chiave}: {valore}')
```

Quando Usare i Dizionari?



- Quando hai bisogno di una struttura dati che associ chiavi a valori.
- Per rappresentare dati strutturati (es. JSON, configurazioni, database in memoria)
- Quando è necessario un accesso rapido ai dati tramite chiavi uniche

Esempio pratico: Memorizzare informazioni sugli studenti di una classe



```
studenti = {  
    'ID001': {'nome': 'Alice', 'voto': 28},  
    'ID002': {'nome': 'Luca', 'voto': 30},  
}  
print(studenti['ID001']['voto']) # Output: 28
```

Conclusione



*I dizionari sono una delle strutture dati più potenti in Python.
Sono altamente efficienti e versatili, rendendoli una scelta eccellente
per la gestione di dati strutturati.*



Gestione delle Date e del Tempo in Python (DateTime)

Il modulo datetime in Python consente di lavorare con date e orari. Questo modulo fornisce classi e metodi per ottenere, manipolare e formattare date e orari.

» Importazione del Modulo datetime

Per lavorare con date e orari in Python, bisogna importare il modulo datetime:

```
import datetime
```

» Ottenere la Data e l'Ora Attuali

```
import datetime  
now = datetime.datetime.now()  
print(now) # Output: 2025-02-08 15:30:45
```



Gestione delle Date e del Tempo in Python (DateTime)

» Creazione di una Data o un'Orario Personalizzato

```
custom_date = datetime.date(2025, 12, 25)  
print(custom_date) # Output: 2025-12-25
```

» Formattazione delle Date e degli Orari

```
now = datetime.datetime.now()  
formatted_date = now.strftime('%d-%m-%Y %H:%M:%S')  
print(formatted_date) # Output: 08-02-2025 15:30:45
```



Gestione delle Date e del Tempo in Python (DateTime)

» Operazioni con Date e Orari

```
from datetime import datetime, timedelta  
oggi = datetime.now()  
domani = oggi + timedelta(days=1)  
print(domani) # Data di domani
```

» Differenza tra Due Date

```
data1 = datetime(2025, 12, 25)  
data2 = datetime(2025, 1, 1)  
differenza = data1 - data2  
print(differenza.days) # Output: 358 giorni
```



Gestione delle Date e del Tempo in Python (DateTime)

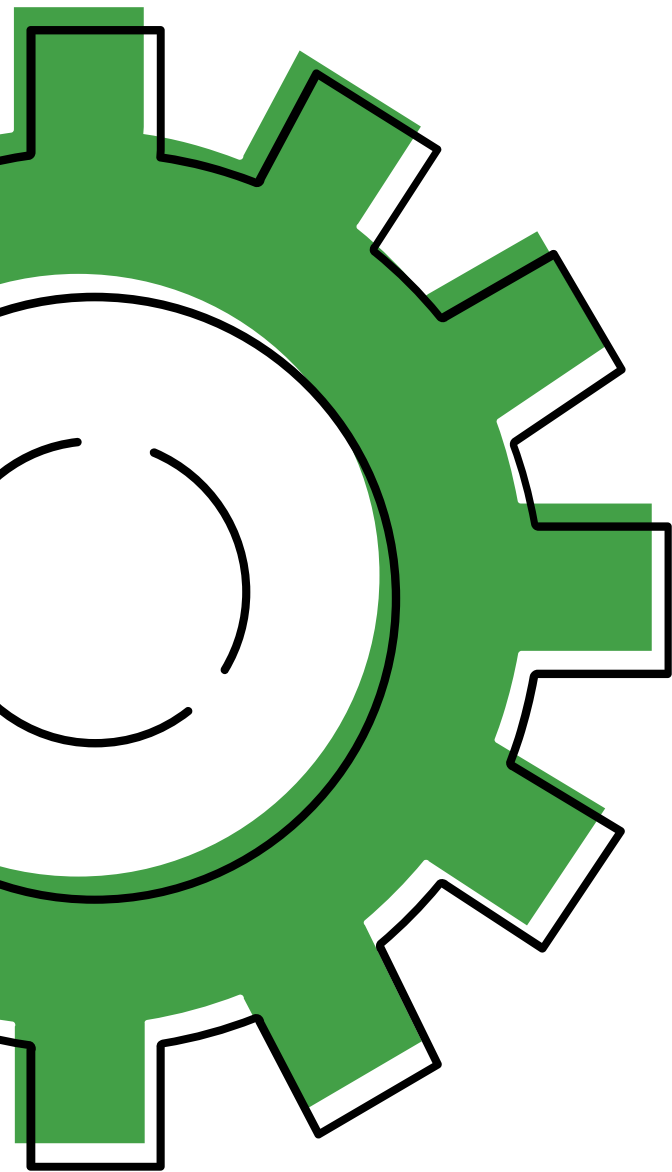
» Ottenere il Giorno della Settimana

```
oggi = datetime.today()  
print(oggi.strftime('%A')) # Output: nome del giorno
```

» Conclusione

Il modulo datetime è essenziale per gestire date e orari in Python. Consente di ottenere, manipolare e formattare date con facilità.

Definire una Funzione in Python



In Python, le funzioni si definiscono con la parola chiave ``def``

La sintassi base è:

```
"""python
def nome_funzione(parametri):
    # Corpo della funzione
    return valore_opzionale
"""
```

Una funzione può accettare parametri e restituire un valore



Esempio di Definizione e Chiamata di Funzione

Esempio di funzione che somma due numeri:

```
```python
def somma(a, b):
 return a + b
```

```
Chiamata della funzione
risultato = somma(5, 3)
print(risultato) # Output: 8
```
```

{ `a` e `b` sono parametri

{ `return` restituisce il valore della somma

Tipologie di Parametri nelle Funzioni



» **Posizionali**: vengono passati nell'ordine definito

```
python
def saluto(nome, cognome):
    print(f'Ciao {nome} {cognome}')
saluto('Mario', 'Rossi')
```

» **Default**: valori predefiniti se non specificati

```
python
def saluto(nome='Utente'):
    print(f'Ciao {nome}')
saluto() # Output: Ciao Utente
```

» **Arbitrari (*args e **kwargs)**:

vengono passati nell'ordine definito

```
python
def somma(*numeri):
    return sum(numeri)
print(somma(1, 2, 3, 4)) # Output: 10
```

Grazie

